

Optimization of AI SoC with Compiler-assisted Virtual Design Platform

Chih-Tsun Huang
National Tsing Hua University
Hsinchu, Taiwan
Industrial Technology Research
Institute
Hsinchu, Taiwan
cthuan@cs.nthu.edu.tw

Juin-Ming Lu
Industrial Technology Research
Institute
Hsinchu, Taiwan
National Tsing Hua University
Hsinchu, Taiwan
jm@itri.org.tw

Yao-Hua Chen*
Industrial Technology Research
Institute
Hsinchu, Taiwan
YaoHuaChen@itri.org.tw

Ming-Chih Tung*
Industrial Technology Research
Institute
Hsinchu, Taiwan
KerwinTung@itri.org.tw

Shih-Chieh Chang
Industrial Technology Research
Institute
Hsinchu, Taiwan
National Tsing Hua University
Hsinchu, Taiwan
shihchieh.chang@itri.org.tw

ABSTRACT

As deep learning keeps evolving dramatically with rapidly increasing complexity, the demand for efficient hardware accelerators has become vital. However, the lack of software/hardware co-development toolchains makes designing AI SoCs (artificial intelligent system-on-chips) considerably challenging. This paper presents a compiler-assisted virtual platform to facilitate the development of AI SoCs from the early design stage. The electronic system-level design platform provides rapid functional verification and performance/energy analysis. Cooperating with the neural network compiler, AI software and hardware can be co-optimized on the proposed virtual design platform. Our Deep Inference Processor is also utilized on the virtual design platform to demonstrate the effectiveness of the architectural evaluation and exploration methodology.

CCS CONCEPTS

• **Hardware** → **Electronic design automation; Very large scale integration design.**

KEYWORDS

deep neural networks, system-on-chip, hardware accelerator, electronic system-level, ESL, virtual platform, AI compiler

ACM Reference Format:

Chih-Tsun Huang, Juin-Ming Lu, Yao-Hua Chen, Ming-Chih Tung, and Shih-Chieh Chang. 2023. Optimization of AI SoC with Compiler-assisted Virtual

*Both authors contributed equally to this research.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from [permissions@acm.org](https://permissions.acm.org).

ISPD '23, March 26–29, 2023, Virtual Event, USA

© 2023 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 978-1-4503-9978-4/23/03...\$15.00
<https://doi.org/10.1145/3569052.3578930>

Design Platform. In *Proceedings of the 2023 International Symposium on Physical Design (ISPD '23)*, March 26–29, 2023, Virtual Event, USA. ACM, New York, NY, USA, 7 pages. <https://doi.org/10.1145/3569052.3578930>

1 INTRODUCTION

Artificial intelligence (AI) has advanced significantly with modern deep neural networks (DNNs) [6, 8] in a variety of applications, such as machine vision, natural language processing, media analytics, content synthesis, and extended reality, etc. DNNs have enormous computation complexity with a large amount of data, resulting in an increasing demand for efficient hardware accelerators. There are a number of state-of-the-art DCNN accelerators [3, 4, 9, 13, 22]. Among them, Eyeriss [3] presents the row-stationary (RS) dataflow based on a 2D PE (Processing Element) array. The RS dataflow has the advantage of reusing data in PE's scratchpads, thus minimizing external data movement. However, the neural network structure of different layers (e.g., the number of input/output channels, kernel sizes, feature map sizes, etc.) can be diversified. As a result, the evaluation of dataflow and its hardware mapping has become a burden due to manual design effort. Several approaches have been published to design the DNN accelerators with the aid of fast performance and power prediction models for different dataflows [10, 11, 15–19]. Besides, improved dataflow with hierarchical on-chip interconnection and PE array were evaluated to reach high data reuse, and then better PE utilization for DCNNs [4, 12]. To support different tile sizes and loop orders specifically for video understanding, the Morph architecture with PE clusters and a three-level buffer hierarchy was recently presented in [7]. In addition, a framework called TENET was presented to utilize a relation-centric notation to represent hardware dataflow, PE interconnection, and data assignment uniformly as relations. An analytical framework was also used to estimate data reuse, bandwidth requirements, latency, and energy [14].

Most of these hardware-centric approaches focus on highly-engineered and design-specific hardware optimization. To enable

both high-level graph optimization and low-level operator optimization for diverse hardware back-ends, TVM compiler [1] was proposed with a tensor expression language and a learning-based optimization framework, AutoTVM, which is a template-guided search scheme to optimize tensor programs for deep learning workloads [2]. Later, an auto-scheduling approach called Ansor [23] was presented to explore a greater amount of optimization combinations by sampling programs from a hierarchical representation of search space. The sampled programs are then fine-tuned with an evolutionary search algorithm. However, these existing auto-tuners perform design space exploration of efficient tensor programs with opaque hardware details. Therefore, the resultant performance can be limited. In [20], Bolt implements a tuner to optimize the parameters for CUTLASS libraries, given the modularized vendor library with reconfigurability via declarative control, bridging the gap between automation and hardware-native performance.

The design of customized AI SoCs (System-on-Chips) and its optimization is even more challenging. As deep learning keeps evolving dramatically, the gap between software algorithms and hardware accelerators widens rapidly. Neither the hardware-centric evaluation tool nor the software-centric auto-tuner can fulfill the urgent demand for an AI design toolchain for AI SoC development. Unlike the standardized IC design flow, there is no standardized methodology for developing deep learning accelerators, their software, and the system application because a standardized software/hardware interface restricts the flexibility and extendibility of hardware accelerators for fast-evolving software algorithms. In contrast to providing a standardized design flow, we propose a compiler-assisted virtual design platform for the software/hardware co-optimization of AI SoC:

- The virtual design platform implements an *Electronic System-Level* (ESL) model of our *Deep Inference Processor* (DIP) SoC, allowing system-level functional verification for deep learning applications.
- The NNMap compiler is built on top of TVM with a smart-scheduling algorithm based on improved template auto-generation. Cooperating with the ESL hardware platform, the turnaround time of the auto-tuner can be significantly reduced. In addition, the NNMap supports multiple accelerator engines inside the SoC, enabling heterogeneous computing of DNNs.
- The design exploration and architectural optimization can be efficient on our virtual design platform with fast and accurate performance and power estimation models. Hardware execution can be profiled in the early stages of design, leading to effective architectural decisions.
- The experiment results show that the NNMap can enhance the inference latency with a speedup of 1.85× on average for various DCNNs.
- We also propose a multi-channel DMA architecture to improve the data transfer. The four-channel scheme achieves a speedup of 2.87×.

With our compiler-assisted virtual design platform, SoC architects can optimize AI acceleration for different needs, e.g., model accuracy, performance, energy efficiency, etc.

2 PROPOSED COMPILER-ASSISTED VIRTUAL PLATFORM FOR AI SOCS

Fig. 1 illustrates the overall architecture of our compiler-assisted virtual design platform. State-of-the-art deep learning frameworks such as Pytorch or TensorFlow can be the front end of our *NN Compiler Framework* to train and fine-tune deep neural network models. The NN Compiler Framework consists of three main components. The *NN Profiler* provides an integrated user interface to analyze deep learning workloads. The *NNArch* is our exploration tool which can evaluate different architectural parameters and optimize the design scheduling for output-stationary and row-stationary DCNN accelerators with fast and accurate analytical performance/energy models [21]. With the help of the NNArch, structure model reduction, e.g., pruning and quantization, can be accomplished effectively with realistic hardware considerations. The *NNMap* is built on top of TVM compiler to cooperate with the NNArch, generating optimized codes for heterogeneous deployable AI accelerators.

The right-hand side of Fig. 1 shows the *Electronic System-Level* (ESL) platform of AI SoCs, also called the NNSim. The ESL platform describes the hardware architecture of our *Deep Inference Processor* (DIP) with *Transaction-Level Modeling* (TLM). The DIP is an AI SoC that consists of the host processor for software and operating system, interface modules such as UART, USB, PCIe, etc., DMA and DRAM controller for data transfer, and other application-specific engines such as video coder/encoder or audio processing unit for accelerating system-level applications. In addition, the DIP also integrates the Deep Learning Accelerator (DLA), or our *NN Accelerator* (NNACC), which will be discussed later. The ESL platform provides functional verification, performance evaluation, and power/energy estimation based on hardware characteristics. It is a powerful tool for exploring architectures and design parameters.

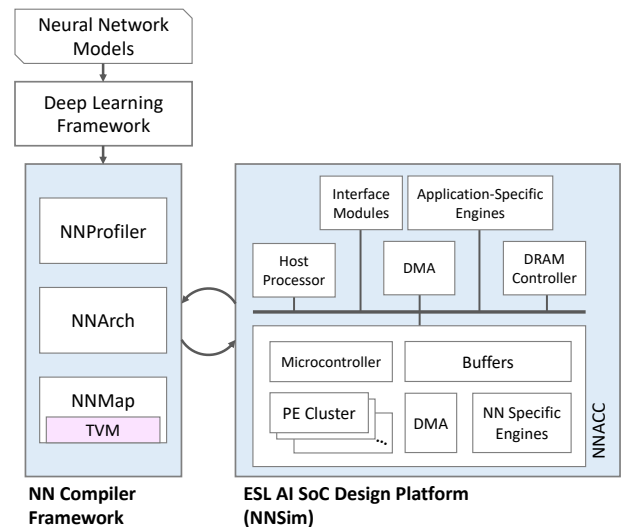


Figure 1: The proposed compiler-assisted virtual platform for AI SoCs.

3 NNMAP: THE ENHANCED AI COMPILER FOR HETEROGENEOUS COMPUTING

Fig. 2 illustrates our deep learning compiler stack. Its front end converts the high-level graphs into Relay intermediate representation (Relay IR). The NNMap adopts TVM's two-level optimization: high-level graph optimization analyzes the computation and dependency of operators, and low-level tensor optimization for scheduling and code generation. Our enhanced compiler partitions the computation graph into multiple subgraphs based on hardware-aware partitioning to support multiple devices for heterogeneous computing.

Conventionally, computational graphs are used to represent deep learning models. Graph-level optimization focuses on operator fusion, constant folding, memory allocation, and data padding within a single device. We extend its support to heterogeneous devices by enhancing the graph partitioning technique. The existing approaches cannot resolve the dependency deadlock that may occur when a computation graph is decomposed into several subgraphs with multiple outputs. Our approach can effectively break the cyclic dependency, creating deployable subgraphs for multiple CPUs, DSPs, and DLAs.

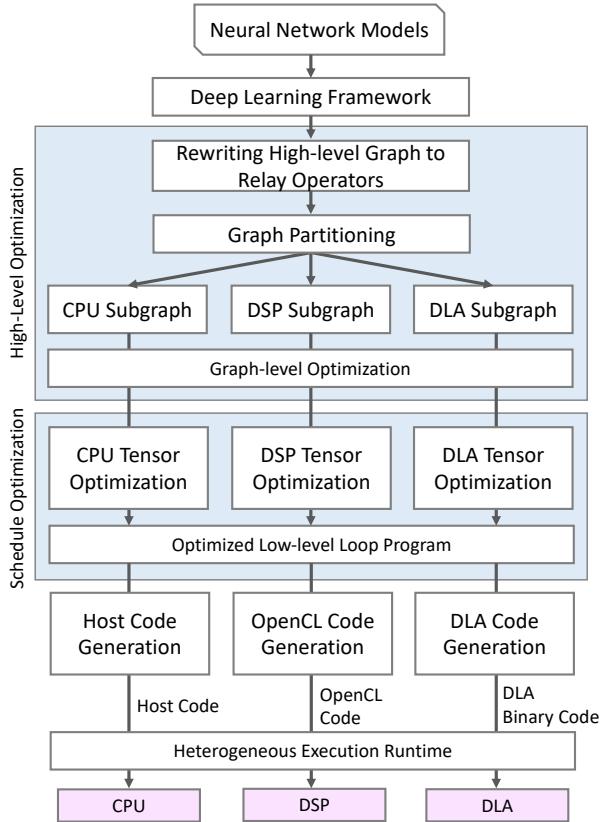


Figure 2: The NNMap compiler stack.

As a result, different devices have their own back-end schedulers, performing customized scheduling algorithms for optimization.

Therefore, hardware-specific code generators can generate source code, object files, or binary executables for specific target devices. In the end, heterogeneous runtime is utilized to pack and deploy the generated codes to different target devices for parallel execution.

3.1 Improved Smart-scheduling technique

The AutoTVM technique [2] defines a template-guided search space of the scheduling for each operator in the TVM compiler. This manual involved search space is limited and less efficient. Therefore, the auto-scheduling technique, also called Ansor, was presented to utilize a hierarchical search space [23]. In addition, Ansor introduces an improved task scheduling and evolutionary search algorithm.

In our compiler stack, we enhance the template generation by defining different tuning knobs of hardware architectural parameters (e.g., shared memory, local memory, threads per block, indexing dimension, vectorization, multi-cores, etc.) based on engineers' expertise. By specifying the hardware configuration, we can automatically generate the schedule template for each operator and cooperate with the auto-tuner to obtain better scheduling.

Fig. 3 compares the scheduling results between our smart-scheduling and TVM baseline for various deep convolutional neural networks (DCNNs) on Intel CPU and NVIDIA RTX2060, respectively. Our approach achieves 1.25× to 3.64× speedup. The latency improvement is 85% on average in this experiment.

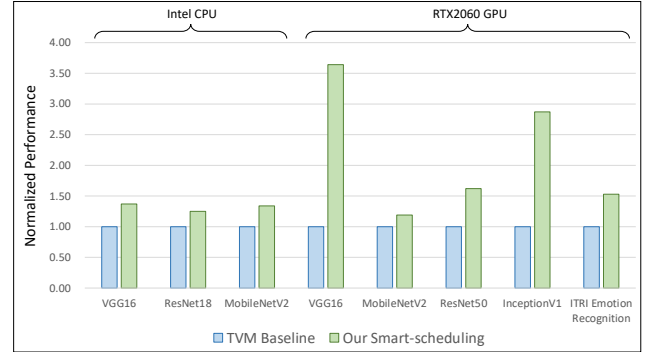


Figure 3: Normalized performance with our smart-scheduling as compared with TVM baseline for different DCNNs.

4 ESL DESIGN PLATFORM FOR AI SOCS

The virtual design platform in Fig. 1 helps verify hardware functionality and validate system integration with software. In addition, system architects can trade off the accuracy of deep learning algorithms, hardware cost, performance, and energy consumption in the early design phase.

4.1 The Convolution Dataflow

Table 1 summarizes the shape parameters of a convolutional layer in the DCNN. Three types of data can be represented as four-dimension tensors: *input feature maps (ifmap)* as $N \times C \times H \times W$; *filter weights (filter)* as $M \times C \times R \times S$; *output feature maps (ofmap)* as $N \times M \times E \times F$, respectively. The naïve process to compute a

single convolutional layer can be expressed by seven nested for-loops, as shown in Fig. 4(a). The outer four for-loops $\{n, m, f, e\}$ walk through all the ofmap pixels. The inner three for-loops $\{r, s, c\}$ are to access the 3-D filters for the *ifmap*. Due to the limitation of hardware resources, our tile-based row-stationary (RS) dataflow [5] partitions the batch size (N), ofmap height (E), output channel dimensions (M), and input channel dimensions (C) into smaller tiles, respectively (see Fig. 4(b) [5]). The tile-based dataflow is therefore divided into two parts: temporal execution and spatial computation. The parallel computation in one processing pass can be mapped to multiple *Processing Element* (PE) clusters, where one PE computes a tiled 1-D convolution.

Table 1: Shape Parameters of Convolutional Layers

Shape Parameter	Description
N	Batch size of 3D ifmaps
M	Number of 3D filters / number of ofmap channels
C	Number of ifmap/filter channels
H/W	Ifmap plane height/width
R/S	Filter plane height/width
E/F	Ofmap plane height/width
U	Convolution stride

4.2 Configurable Tile-based Architecture for DNN Acceleration

Fig. 5 shows our scalable NNACC architecture for DCNNs [5]. The PE array consists of u PE Cluster Groups. Each PE Cluster Group is equipped with one Global Buffer Bank to deliver the *ifmap*, *filter* into the row-wise PE clusters. The *psum* or *ofmap* are accumulated in a column-wise manner into k global buffer banks. Data is scattered and gathered through pipelining.

Our PE cluster implements the modified row-stationary (RS) scheme from [3] with a 2-D PE array. The cluster also has intra-PE networks to deliver the *ifmap*, *filter*, input partial sums (*ipsu*) and to collect the output partial sums (*opsum*). Each cluster has a cluster controller to execute processing passes for tiled computation. Each PE performs multiplication and accumulation. To maximize data use, three different scratchpad memories have been designed for *ifmap*, *filter*, and *psum*, respectively. Input buffers (IBuf) and output buffers (OBuf) are also used to convert bandwidth mismatches between PE and inter-PE networks.

4.3 The NN Architectural Evaluation Tool

With the basic architecture of our tiled-based DNN accelerator, a design space and architectural parameter exploration tool, NNArch, is proposed [21] to generate optimized design scheduling for row-stationary DNN accelerators with fast and accurate analytical performance/energy models. Given target DNN models, designers start with the initial candidates for tile selections and data bandwidths. The NNArch searches for possible combinations of different architectural parameters for design trade-offs. With the parameter configuration, the target DNN is then tiled and scheduled based on our NNACC with optimized PPA (performance, power, and area).

Naïve 7 For-loops Convolution

```
for (n=0; n<N; n++) {
  for (m=0; m<M; m++) {
    for (f=0; f<F; f++) {
      for (e=0; e<E; e++) {
        for (r=0; r<R; r++) {
          for (s=0; s<S; s++) {
            for (c=0; c<C; c++) {
              ofmap += ifmap x filter;
            }
          }
        }
      }
    }
  }
}
```

(a)

Tile-Based Row-Stationary Dataflow

<pre>for (n0=0; n0<N; n0+=n) { for (e0=0; e0<E; e0+=e) { for (g0=0; g0<G; g0+=g) { for (m0=0; m0<M; m0+=m) { for (c0=0; c0<C; c0+=q*r) { for (m1=0; m1<m; m1+=p*t) {</pre>	Temporal Execution	
<pre>// ----- Processing Pass ----- // PE Array Level for (g2=0; g2<g; g2++) { for (c2=0; c2<r; c2++) { for (m2=0; m2<t; m2++) { // PE Set Level for (r3=0; r3<R; r3++) { for (e3=0; e3<e; e3++) { // PE Level for (n4=0; n4<n; n4++) { for (f4=0; f4<F; f4++) { for (s4=0; s4<S; s4++) { for (c4=0; c4<q; c4++) { for (m4=0; m4<p; m4++) { ofmap += ifmap x filter; } } } } } } } } } }</pre>	Tile-Base Mapping RS 1D Convolution in Each PE	Spatial Execution

(b)

Figure 4: (a) naïve seven for-loops representation for a convolution layer; (b) our tile-based dataflow [5].

4.4 ESL Platform of DIP

Fig. 6 illustrates the ESL platform to model our DIP. The high-level rapidly-generated functional models of individual blocks are connected by Transaction-Level Modeling (TLM) for fast architectural evaluation. With the help of the analytical timing estimation models from NNArch, the ESL platform can provide cycle-accurate timing simulation at the SystemC level. In addition, there are also approximate models for area and power consumption with parameters extracted from our previous simulation and measurement results of fabricated test chips. The estimation models are fed back to the ESL platform. A *Performance/Power Counter* is attached to each hardware module to collect performance and power information during the simulation. The ESL platform allows the NNMap compiler to generate optimized code and simulates the SW/HW execution during the development phase. Even after the hardware is fabricated, the integrated compiler-assisted ESL platform can significantly reduce the turnaround time of the auto-tuning.

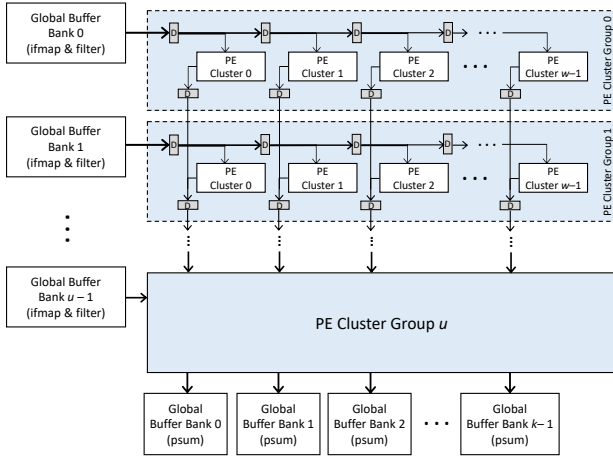


Figure 5: The architecture of our tile-based DCNN accelerator [5].

Note that in this figure, this example platform consists of individual engines for different NN operators (e.g., convolution, activation, pooling, etc.). Note that operator fusion is possible based on either the designer's intuition or the analysis of the system-level simulation.

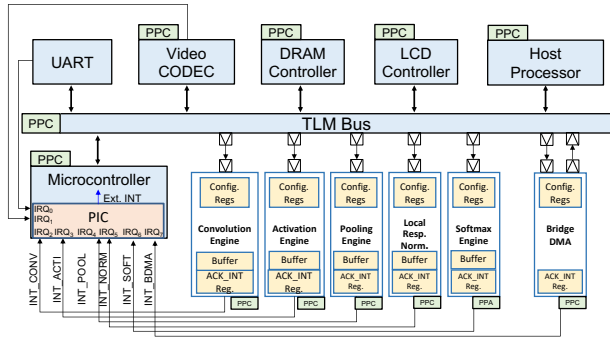


Figure 6: The architecture of the proposed ESL platform for a specific DIP instance.

With timing and power estimation models from the NNArch, the ESL platform provides fast and accurate performance analysis. This allows us to perform effective profiling and performance analysis for individual hardware and software components. Fig. 7 shows the layer-by-layer simulation of AlexNet executed on the NNACC.

Fig. 8(a) summarizes the latency breakdown of different layers in ResNet50 on the NNACC. Data transfer time (e.g., transferring the ofmap, bias, filter weights, ID of individual PE, ifmap, etc.) and computation time can be analyzed. In this experiment, we relax one specific tiling constraint for a possible latency improvement (see Fig. 8(b)). With the cooperation of NNArch and NNSim, we can effectively evaluate the performance bottleneck and potential improvement. This figure shows that the new tiling scheme does not

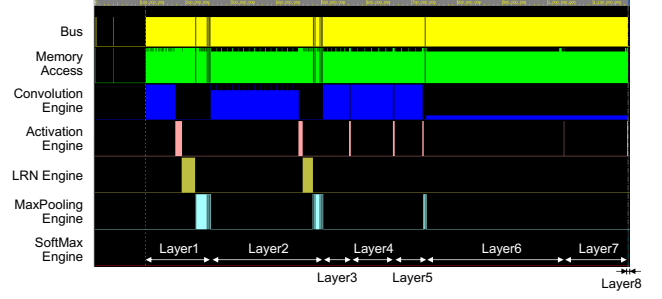


Figure 7: Timing analysis for individual hardware blocks with AlexNet.

improve some specific layers due to their neural network structures (e.g., the 1-3 layers, etc.). Also, the relaxed tiling scheme comes with the extra overhead of hardware control. With our ESL platform,

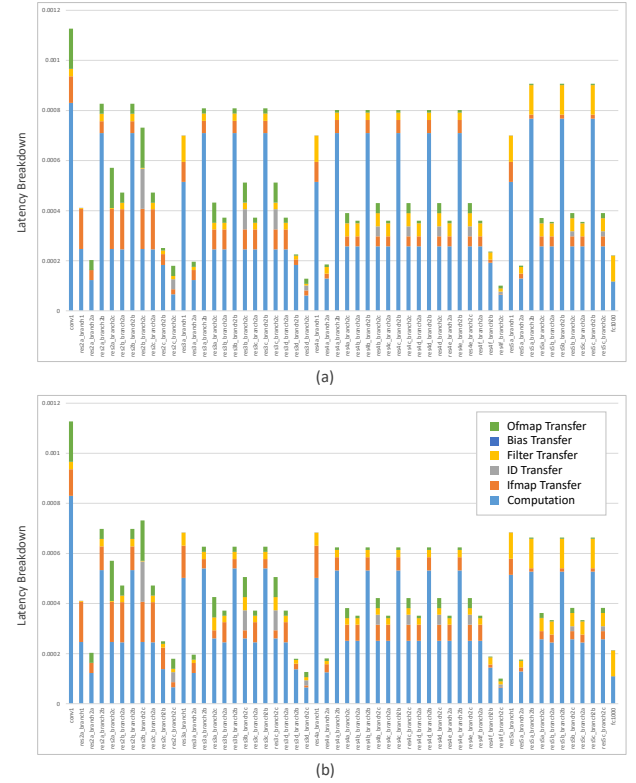


Figure 8: Profiling and optimization of design parameters (a) with the baseline tiling and scheduling; (b) with a relaxed tiling for a better performance.

In addition to speeding up the computation, data movement is essential in executing DNNs. Fig. 9 illustrates a multi-channel DMA architecture with the multi-bank DRAM cluster on our ESL platform. The abstraction of the memory subsystem with interconnection and sequencers is shown in Fig. 10. The ESL simulation

is able to identify that both data and control commands can be reordered to balance memory transfer. The abstraction serves as an adequate representation to develop the data-dispatching algorithm for multiple DMA channels based on monitoring the incoming workload.

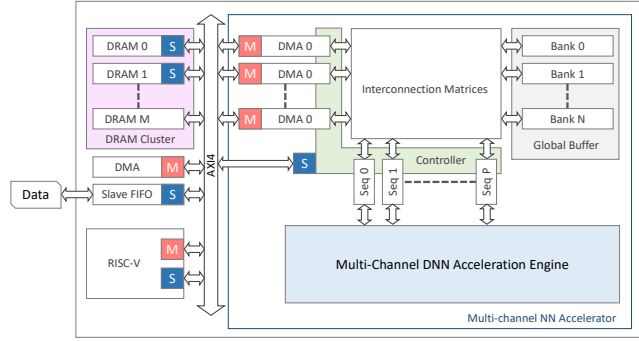


Figure 9: Multi-channel DMA architecture on the ESL platform.

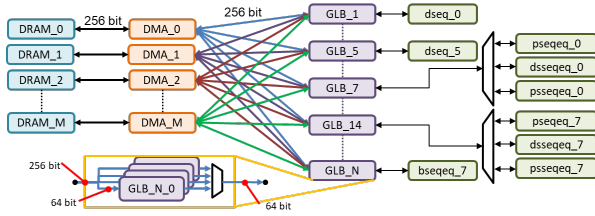


Figure 10: Multi-channel abstraction model.

The experiment result shows that the performance of the four-channel architecture achieves a $2.87\times$ speedup compared with the single-channel baseline. Our multi-channel reordering scheme can improve the two-channel design by 15.9% and the four-channel design by 19.2%, respectively (see Fig. 11).

5 CONCLUSION

We present a compiler-assisted virtual platform to shorten the development cycle of AI SoCs. The NN Compiler framework adopts the state-of-the-art TVM compiler to provide optimized code for heterogeneous devices with a smart-scheduling technique. The ESL design platform implements our proprietary DIP, allowing rapid functional verification and performance/energy evaluation at the system level. By using the proposed AI SoC toolchain, system architects will be able to co-optimize software and hardware in the early development stages.

REFERENCES

- [1] Tianqi Chen, Thierry Moreau, Ziheng Jiang, Lianmin Zheng, Eddie Yan, Haichen Shen, Meghan Cowan, Leyuan Wang, Yuwei Hu, Luis Ceze, Carlos Guestrin, and Arvind Krishnamurthy. 2018. {TVM}: An automated {End-to-End} optimizing

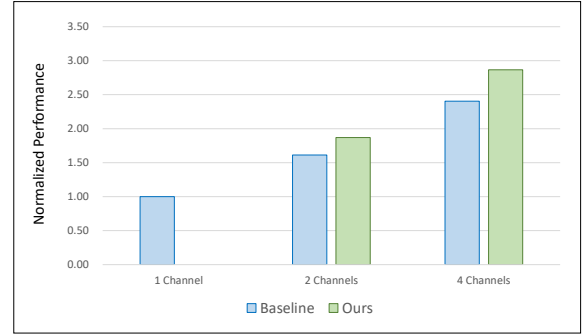


Figure 11: Comparison on channel utilization with the 1-channel, 2-channel, and 4-channel DMA architectures.

- compiler for deep learning. In *13th USENIX Symposium on Operating Systems Design and Implementation (OSDI 18)*. 578–594.
- [2] Tianqi Chen, Lianmin Zheng, Eddie Yan, Ziheng Jiang, Thierry Moreau, Luis Ceze, Carlos Guestrin, and Arvind Krishnamurthy. 2018. Learning to optimize tensor programs. *Advances in Neural Information Processing Systems* 31 (2018).
- [3] Yu-Hsin Chen, Tushar Krishna, Joel S Emer, and Vivienne Sze. 2016. Eyeriss: An energy-efficient reconfigurable accelerator for deep convolutional neural networks. *IEEE journal of solid-state circuits* 52, 1 (2016), 127–138.
- [4] Yu-Hsin Chen, Tien-Ju Yang, Joel Emer, and Vivienne Sze. 2019. Eyeriss v2: A flexible accelerator for emerging deep neural networks on mobile devices. *IEEE Journal on Emerging and Selected Topics in Circuits and Systems* 9, 2 (2019), 292–308.
- [5] Yang-Tsai Chen, Yu-Xiang Yen, Chun-Tse Chen, Tzu-Yu Chen, Chih-Tsun Huang, Jing-Jia Liou, and Juin-Ming Lu. 2021. Tile-based architecture exploration for convolutional accelerators in deep neural networks. In *2021 IEEE 3rd International Conference on Artificial Intelligence Circuits and Systems (AICAS)*. IEEE, 1–4.
- [6] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. 2016. Deep residual learning for image recognition. In *Proceedings of the IEEE conference on computer vision and pattern recognition*. 770–778.
- [7] Kartik Hegde, Rohit Agrawal, Yulun Yao, and Christopher W Fletcher. 2018. Morph: Flexible acceleration for 3d cnn-based video understanding. In *2018 51st Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*. IEEE, 933–946.
- [8] Andrew G Howard, Menglong Zhu, Bo Chen, Dmitry Kalenichenko, Weijun Wang, Tobias Weyand, Marco Andreetto, and Hartwig Adam. 2017. Mobilenets: Efficient convolutional neural networks for mobile vision applications. *arXiv preprint arXiv:1704.04861* (2017).
- [9] Norman P Jouppi, Cliff Young, Nishant Patil, David Patterson, Gaurav Agrawal, Raminder Bajwa, Sarah Bates, Suresh Bhatia, Nan Boden, Al Borchers, et al. 2017. In-datacenter performance analysis of a tensor processing unit. In *Proceedings of the 44th annual international symposium on computer architecture*. 1–12.
- [10] Liu Ke, Xin He, and Xuan Zhang. 2018. Nnest: Early-stage design space exploration tool for neural network inference accelerators. In *Proceedings of the International Symposium on Low Power Electronics and Design*. 1–6.
- [11] Hyoukjun Kwon, Michael Pellauer, and Tushar Krishna. 2018. MAESTRO: an open-source infrastructure for modeling dataflows within deep learning accelerators. *arXiv preprint arXiv:1805.02566* (2018).
- [12] Hyoukjun Kwon, Ananda Samajdar, and Tushar Krishna. 2018. Maeri: Enabling flexible dataflow mapping over dnn accelerators via reconfigurable interconnects. *ACM SIGPLAN Notices* 53, 2 (2018), 461–475.
- [13] Jinmook Lee, Changhyeon Kim, Sanghoon Kang, Dongjoo Shin, Sangyeob Kim, and Hoi-Jun Yoo. 2018. UNPU: A 50.6 TOPS/W unified deep neural network accelerator with 1b-to-16b fully-variable weight bit-precision. In *2018 IEEE International Solid-State Circuits Conference (ISSCC)*. IEEE, 218–220.
- [14] Liqiang Lu, Naiqing Guan, Yuyue Wang, Liancheng Jia, Zizhang Luo, Jieming Yin, Jason Cong, and Yun Liang. 2021. Tenet: A framework for modeling tensor dataflow based on relation-centric notation. In *2021 ACM/IEEE 48th Annual International Symposium on Computer Architecture (ISCA)*. IEEE, 720–733.
- [15] Anghuman Parashar, Priyanka Raina, Yakun Sophia Shao, Yu-Hsin Chen, Victor A Ying, Anurag Mukkara, Rangharajan Venkatesan, Bruce Khailany, Stephen W Keckler, and Joel Emer. 2019. Timeloop: A systematic approach to dnn accelerator evaluation. In *2019 IEEE international symposium on performance analysis of systems and software (ISPASS)*. IEEE, 304–315.

- [16] Brandon Reagen, Paul Whatmough, Robert Adolf, Saketh Rama, Hyunkwang Lee, Sae Kyu Lee, José Miguel Hernández-Lobato, Gu-Yeon Wei, and David Brooks. 2016. Minerva: Enabling low-power, highly-accurate deep neural network accelerators. In *2016 ACM/IEEE 43rd Annual International Symposium on Computer Architecture (ISCA)*. IEEE, 267–278.
- [17] Ananda Samajdar, Jan Moritz Joseph, Yuhao Zhu, Paul Whatmough, Matthew Mattina, and Tushar Krishna. 2020. A systematic methodology for characterizing scalability of dnn accelerators using scale-sim. In *2020 IEEE International Symposium on Performance Analysis of Systems and Software (ISPASS)*. IEEE, 58–68.
- [18] Ananda Samajdar, Yuhao Zhu, Paul Whatmough, Matthew Mattina, and Tushar Krishna. 2018. Scale-sim: Systolic cnn accelerator simulator. *arXiv preprint arXiv:1811.02883* (2018).
- [19] Yannan Nellie Wu, Joel S Emer, and Vivienne Sze. 2019. Accelergy: An architecture-level energy estimation methodology for accelerator designs. In *2019 IEEE/ACM International Conference on Computer-Aided Design (ICCAD)*. IEEE, 1–8.
- [20] Jiarong Xing, Leyuan Wang, Shang Zhang, Jack Chen, Ang Chen, and Yibo Zhu. 2022. Bolt: Bridging the Gap between Auto-tuners and Hardware-native Performance. *Proceedings of Machine Learning and Systems* 4 (2022), 204–216.
- [21] Che-Chang Yang, Yung-Tai Shih, Chun-Chen Chen, Chih-Tsun Huang, Jing-Jia Liou, Yao-Hua Chen, and Juin-Ming Lu. 2022. Efficient Segment-wise Pruning for DCNN Inference Accelerators. In *2022 International Symposium on VLSI Design, Automation and Test (VLSI-DAT)*. IEEE, 1–4.
- [22] Shijin Zhang, Zidong Du, Lei Zhang, Huiying Lan, Shaoli Liu, Ling Li, Qi Guo, Tianshi Chen, and Yunji Chen. 2016. Cambricon-X: An accelerator for sparse neural networks. In *2016 49th Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*. IEEE, 1–12.
- [23] Lianmin Zheng, Chengfan Jia, Minmin Sun, Zhao Wu, Cody Hao Yu, Ameer Haj-Ali, Yida Wang, Jun Yang, Danyang Zhuo, Koushik Sen, Joseph E. Gonzalez, and Ion Stoica. 2020. Ansor: Generating {High-Performance} Tensor Programs for Deep Learning. In *14th USENIX symposium on operating systems design and implementation (OSDI 20)*. 863–879.